CardioNary: Chest X-ray Model Roadmap

Working plan for the lung team, Monday 26 October to Tuesday 17 November 2026

CS F407 Artificial Intelligence · Team CardioNary · Branch `lung` · Internal working document · Revised Thu 12 Nov

1. The plan in one page

Where we are (Mon 26 Oct). Only the baseline exists: the shared code in `core/` and the leakage tests. Nothing has been committed to `lung` yet.

What we deliver by Tue 17 Nov. The chest X-ray branch exactly as the submitted proposal describes it (Sections 4.1 to 4.7), with results, report sections, slides, AI-usage note and demo video all ready. The days after 17 Nov are buffer only.

Changes on Thu 12 Nov. The two pages of the tool become one Streamlit dashboard for both teams, `app/app.py`, built by the heart team from the frontend guide in `docs/STREAMLIT_CLINICAL_FRONTEND_GUIDE.md`. It shows the X-ray and heart results side by side and makes the visit decision (proposal 4.4), so `lung/app_xray.py` is dropped. Vedansh instead writes `lung/predict.py`, the one function the dashboard calls for an X-ray result, and Harsh and Mihir hand over the fitted numbers and model files it needs. All code, the dashboard included, ends on Mon 16 Nov; the write-ups (report and slides) run alongside it from Fri 13 to Tue 17 Nov. Sections 2 to 7 and both figures follow these changes.

Stage	Dates	Who	What we get
S0 Setup and data	Mon 26 to Tue 27 Oct	Everyone, each on their own	A working environment and your own copy of the data you need
S1 Each person's first piece	Tue 27 to Fri 30 Oct → **M1**	Everyone, nobody waits for anybody	Base CNN with its 8-bit copy; characteristics and feature vote; adaptation code; RSNA data, metrics and corruptions
S2 Calibration, metadata vote, late fusion	Mon 2 to Fri 6 Nov → **M2**	Harsh, Vedansh, Mihir	Fusion results on Kermany and RSNA
S3 Test-time adaptation	Mon 2 to Sat 7 Nov	Aayush	How much accuracy adaptation recovers, and how fast
S4 8-bit model, robustness, edge budget	Sat 7 to Tue 10 Nov → **M3**	Mihir, Aayush, Vedansh, Harsh	8-bit model measured on a laptop; corruption results
S5 Code complete, results frozen	Wed 11 to Mon 16 Nov → **M4**	Everyone	`lung/predict.py` in the dashboard and all X-ray code merged; every number in a committed file
S6 Report, slides and deliverables	Fri 13 to Tue 17 Nov → **M5**	Everyone	Report sections, slides, AI-usage note and demo video, all ready; written alongside S5

M1 falls in the week of the mid-project check-in. Figure 1 shows what each stage builds, and Figure 2 shows the dates.

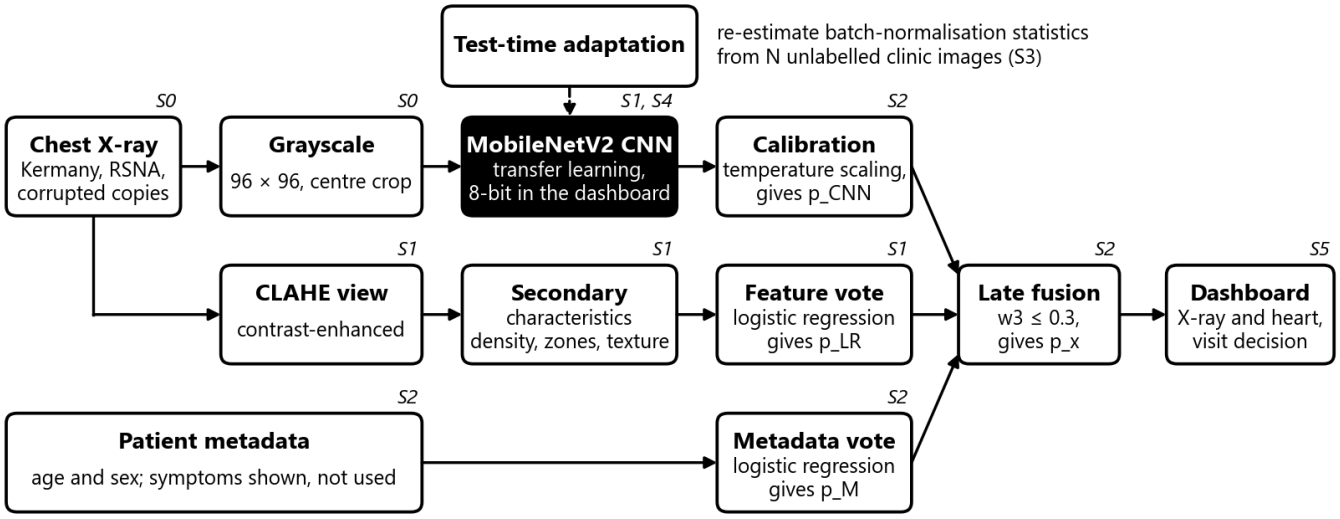


Figure 1. The chest X-ray pipeline, as in the proposal's Figure 1. Each box is labelled with the stage that builds it.

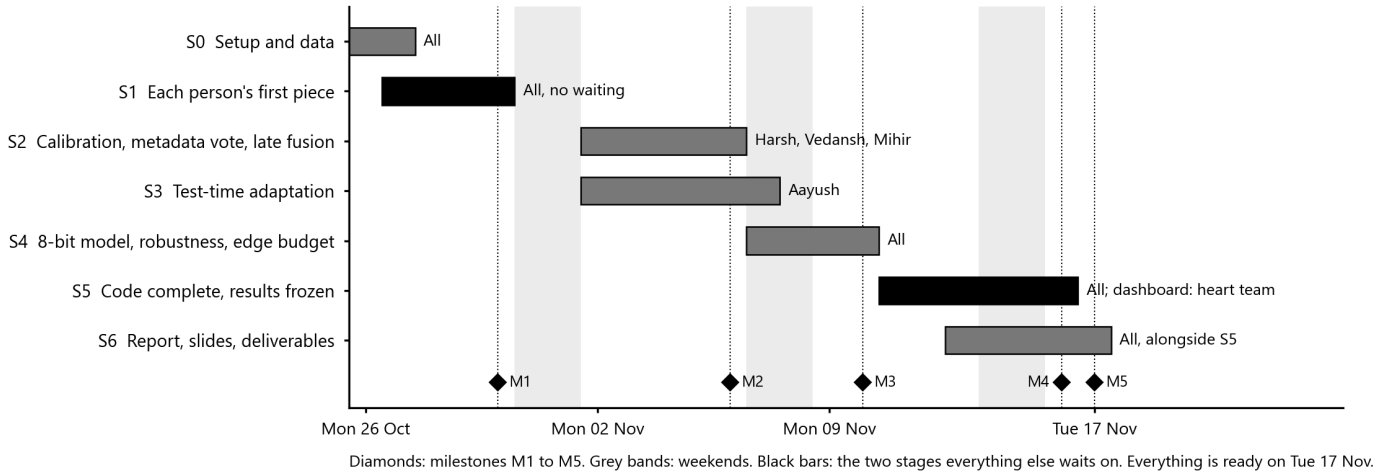


Figure 2. Dated plan. Stages S1 and S5 are on the critical path: later stages need the base CNN, and the final numbers and the demo video need frozen code. All code ends on Mon 16 Nov, including the dashboard, which the heart team builds; the write-ups run alongside from Fri 13 Nov.

2. Choices we have made

Where the proposal left a choice, we take the option that is quickest to build and still meets the proposal's targets.

Topic	Our choice	Why
Kermany copy	Mendeley Data v2, `ChestXRay2017.zip`	Its files match the committed manifest, so the 5 patient-disjoint folds are already fixed
CNN (proposal 4.1)	MobileNetV2 (width 0.35, ImageNet weights) on 96 × 96 grayscale, as already written in `core/train.py`	No new model code; the baseline reached 0.96 to 0.98 per-patient accuracy per fold
Unseen dataset (4.6)	RSNA Pneumonia: "Normal" (8,851) vs "Lung Opacity" (6,012)	One Kaggle download; age and sex are in the file headers
Contrast-enhanced view (4.2)	OpenCV CLAHE, clip limit 2.0, 8 × 8 tiles	One function call
Feature and metadata votes (4.2)	Logistic regression with balanced class weights	Small, fast and easy to explain
Fusion weights (4.2)	Try every $w_1 + w_2 + w_3 = 1$ in steps of 0.05 with $w_3 \le 0.3$; keep the lowest log-loss on the training folds	A few hundred combinations, seconds to run
Calibration (4.4)	Temperature scaling with one number T	About ten lines; the model itself does not change
Adaptation (4.3)	Re-estimate the batch-normalisation statistics in one forward pass	No training, no gradients
8-bit model (4.5)	`core/quantize.py` as it is	Already works: the baseline gave 610 kB and 99.3% agreement
Symptoms (fever, cough)	Asked and shown in the dashboard, not used by the metadata vote	No public X-ray dataset records them, so there is nothing to learn a weight from
Offline tool (4.5)	One Streamlit dashboard, `app/app.py`, for the X-ray and heart results and the visit decision, built by the heart team from the frontend guide; English and Hindi	One screen for the health worker; pure Python; runs without internet
X-ray result for the dashboard	`lung/predict.py`: one call takes an image and the patient details and returns the `ModelOutput` and `FusedOutput` of `core/contracts.py`	The dashboard never touches the model or its weights, as the guide requires
Where code goes	Everything new goes in `lung/`, plus new test files in `tests/`. We only import from `core/` and never change it	The team can work without touching shared code

Left out. VinDr-CXR (the proposal says "if time allows"), the bacterial and viral split, early fusion and Grad-CAM. Entropy minimisation is left out unless S3 finishes early.

3. Tools and data

Tool	Version	Used for
Python	3.10	Everything
TensorFlow, Keras, TensorFlow Lite	2.19.1, 3.12.4	MobileNetV2, adaptation, 8-bit conversion and inference
scikit-learn	1.7.2	Logistic regressions, AUROC
NumPy, SciPy, pandas	2.1.3, 1.15.3, 2.3.3	Arrays, temperature fitting, tables
Pillow, OpenCV (headless)	12.3.0, new	Images, JPEG corruption, CLAHE

Tool	Version	Used for
pydicom	3.0.2	Reading RSNA files, age and sex; DICOM images in `lung/predict.py`
Kaggle command-line tool	new	RSNA download
Streamlit	1.65.0	The dashboard. Later releases need Python 3.11
pytest, GitHub Actions	9.1.1	Tests on every pull request

Dataset	Used for	Size	Labels and metadata	Where to get it
Kermany chest X-rays (Mendeley v2)	Training and 5-fold testing	5,856 images of 2,790 children aged 1 to 5; 1,583 normal, 4,273 pneumonia; 1.2 GB	Normal / pneumonia; no metadata	data.mendeley.com/datasets/rscbjbr9sj/2, CC BY 4.0
RSNA Pneumonia Detection Challenge	Unseen adult dataset; trains and tests the metadata vote	26,684 images, one per patient; we keep 14,863; 3.7 GB	Normal / Lung Opacity; age, sex; view position (never used)	kaggle.com/c/rsna-pneumonia-detection-challenge: join the competition and accept its rules first
Corrupted Kermany copies	Controlled shift (proposal 4.6)	Made from Kermany fold 0 (1,239 images)	Same labels	Our own script

Images never go into Git; only scripts, manifests and small result files do. Kaggle's rules allow sharing RSNA files only with people who have also accepted them, so whoever handles RSNA accepts the rules on their own account.

4. Stage by stage

Initials: **M** Mihir Shitalkumar Joshi, **H** Harsh Ramesh Lambe, **A** Aayush Pratap Asnani, **V** Vedansh Singh. Owners follow the roles in the submitted proposal.

S0 Setup and data (Mon 26 to Tue 27 Oct)

Everyone sets up alone and downloads the data they need themselves.

Everyone

☐ Set up and run the tests:

```
git checkout lung && git pull
python -m venv .venv && .venv\Scripts\activate
pip install -r requirements.txt
pip install pydicom opencv-python-headless kaggle streamlit
python -m pytest tests/
```

Mihir, Harsh and Aayush, each on their own laptop

☐ Download `ChestXRay2017.zip` and unzip it so that `data/raw/kermany/img/chest_xray/train/NORMAL/` exists.

☐ Run `python -m core.build_kermany_manifest`. Then `git diff data/manifests/kermany_manifest.csv` must show no change, which proves your copy matches the committed folds.

☐ Build the image cache: `python -m core.preprocess --dataset kermany`. It writes `data/cache/kermany_logmel.npz`; the name is historical, and the file holds the 96 × 96 images.

Vedansh

☐ Join the RSNA competition on Kaggle, accept the rules, and put your API token in `%USERPROFILE%\.kaggle\kaggle.json`.

☐ Download and unzip: `kaggle competitions download -c rsna-pneumonia-detection-challenge -p data/raw/rsna`.

☐ Commit `lung/requirements-lung.txt` with the four new packages at the versions `pip freeze` shows.

Done when the tests pass for everyone, three people have the Kermany cache, and RSNA is unzipped.

S1 Each person's first piece (Tue 27 to Fri 30 Oct) → M1

Every task in this stage uses only the data you downloaded in S0, so nobody waits for anybody.

Mihir: base CNN and its 8-bit copy (proposal 4.1, 4.5)

☐ Write `lung/model/cv_oof.py`. It runs the same 5-fold loop as `core/train.py`: copy the loop from its `main()`, which calls `run_fold` for each fold. After each fold, run the returned model on that fold's test images, normalised with the returned mean and standard deviation, and keep the probabilities.

- Output: `reports/xray_oof_kermany.csv` with index, patient_id, fold, label, p_normal, p_pneumonia. These "out-of-fold" probabilities come from models that never saw those images, which is what calibration and fusion need.
- Expect about 8 minutes per fold on a laptop CPU.

☐ Per-patient accuracy (mean probability per patient) must be ≥ 90%. The baseline reached 0.96 to 0.98 per fold.

☐ Build the final model and its 8-bit copy, trained on folds 1 to 4 with fold 0 held out:

```
python -m core.quantize --dataset kermany --arch mobilenet --holdout-fold 0 --augment
```

`--augment` is needed because training augments by default but this script does not. The command writes the float model, the 8-bit `.tflite` file and a report with the normalisation mean and standard deviation. Put the two model files in the team's private drive folder by Fri 30 Oct; from S2 on, everyone uses this copy.

Harsh: secondary characteristics and the feature vote (proposal 4.2)

☐ `lung/model/clahe.py`: the CLAHE view of a 96 × 96 image. It is shown in the dashboard and gives three texture numbers for the lung area: mean, standard deviation and mean absolute Laplacian.

☐ `lung/model/features.py`: for every Kermany image in the cache, compute the numeric fields of `vision_descriptors` (in `core/descriptors.py`) plus the three CLAHE numbers. Output: `reports/xray_features_kermany.csv`.

- Numeric fields: dense area, asymmetry index, zone percentages, largest-region fraction, regions detected, mean intensity, texture heterogeneity.
- Text fields such as `pattern` are left out.

☐ `lung/fusion/votes.py`, feature vote: standardise, then logistic regression on the 5 folds already in the Kermany manifest. Output: out-of-fold p_{LR} in `reports/xray_plr_kermany.csv`, and its AUROC on its own.

☐ `lung/fusion/calibrate.py`: fit one temperature T by minimising the negative log-likelihood (`scipy.optimize.minimize_scalar`) on log-probabilities, z = log p. Test it on made-up probabilities: over-confident ones must give T > 1.

Aayush: adaptation code on your own copy of the model (proposal 4.3)

☐ Run the same `core.quantize` command as Mihir (about 10 minutes) to get your own copy of the model. You switch to Mihir's copy in S3.

☐ `lung/tta/bn_adapt.py`, which makes an adapted copy of the model; the original is never changed:

1. Load the float model.
2. Set every batch-normalisation layer, including those inside the nested MobileNetV2, to `trainable = True` and `momentum = 0.0`.
3. Run the N clinic images through it once with `training=True`. Each layer now holds the clinic's mean and variance; no gradients are computed.
4. Set the layers back to `trainable = False` and save the adapted copy.
5. Collapse guard: if the adapted copy labels more than 95% of the N images as one class while the original did not, keep the original statistics.

☐ Try it on a shift you make yourself: Kermany fold 0 made darker (× 0.7) and flatter (contrast halved). Adapt on 32 of those images and score the rest.

☐ `lung/eval/timing.py`: mean time per image and peak memory (`time.perf_counter`, `tracemalloc`) for the float and 8-bit models.

Vedansh: RSNA data, metrics and corruptions (proposal 4.6, 4.7)

☐ `lung/data/build_rsna.py`:

- read `stage_2_detailed_class_info.csv`; keep "Normal" (label 0) and "Lung Opacity" (label 1), one row per patient;
- read age, sex and view position from each file header (`pydicom.dcmread(path, stop_before_pixels=True)`); drop the few images with age above 100, which are known data errors;
- save each image as a 256 × 256 PNG in `data/raw/rsna/png256/`, then build a 96 × 96 cache with `core.images.features_for_image`;
- assign 5 stratified folds. Write `data/manifests/rsna_manifest.csv` (path, patient_id, label, fold, age, sex, view_position) and commit it; RSNA is public and anonymised.

☐ `lung/eval/metrics.py`, with tests in `tests/test_lung_metrics.py` on small made-up arrays:

- per-patient accuracy;
- AUROC;
- sensitivity at 90% specificity;
- expected calibration error with 15 bins;
- 95% bootstrap interval from 1,000 resamples of patients, seed 0.

☐ `lung/eval/corrupt.py`: four corruptions at five severities on 96 × 96 images in [0, 1]:

- blur: σ = 0.5, 1, 1.5, 2, 3 px;
- noise: σ = 0.02, 0.04, 0.06, 0.08, 0.10;
- contrast: x' = (x − mean) · c + mean, c = 0.75, 0.6, 0.45, 0.3, 0.2;

- JPEG: quality 80, 60, 40, 20, 10.

Done when (M1, Fri 30 Oct) all four pieces work:

- the base CNN reaches ≥ 90% per patient, its out-of-fold file exists, and the final model files are in the drive;
- the feature vote has out-of-fold results;
- adaptation works on Aayush's own test shift;
- the RSNA manifest and cache exist, the metric tests pass, and the corruptions run.

S2 Calibration, metadata vote and late fusion (Mon 2 to Fri 6 Nov) → M2

From here on, everyone uses Mihir's final model and out-of-fold file.

Harsh (proposal 4.2, 4.4)

☐ Fit T on `reports/xray_oof_kermany.csv`; report calibration error before and after.

☐ Metadata vote: logistic regression on age/100, $(\text{age}/100)^2$ and sex.

- It is trained and tested only on RSNA, with its 5 folds, which gives out-of-fold p_M.
- View position is never an input: it reflects how a patient was imaged, not the patient.

☐ `lung/fusion/late_fusion.py`: $p = w_1 \cdot p_{CNN} + w_2 \cdot p_{LR} + w_3 \cdot p_M$.

- The weights for fold k are chosen on the other folds only. A missing vote gets weight 0, and the other weights are divided by their sum.
- Return each result as `FusedOutput("xray", ...)` from `core/contracts.py`, which refuses $w_3 > 0.3$.

☐ Fusion on Kermany: CNN plus feature vote; there is no metadata there.

Vedansh (proposal 4.6, 4.7)

☐ `lung/eval/run_rsna.py`: run the final model on all RSNA images, normalised with the Kermany mean and standard deviation, never RSNA's own. Report AUROC, sensitivity at 90% specificity and calibration error for the plain CNN. Also run Harsh's `features.py` on RSNA.

☐ With Harsh, two comparisons with bootstrap intervals in `reports/xray_fusion.json`:

- **Generalisation**, using no RSNA labels: plain CNN against CNN plus feature vote, with weights chosen on Kermany.
- **Metadata gain**, within RSNA folds: CNN plus feature vote, with and without the metadata vote. The weights are re-chosen on the training folds in both cases.

Mihir

☐ Occlusion maps (`vision_zone_attribution` on the 8-bit model) for fold 0 and a sample of RSNA, for the report.

☐ Agreement between the float and 8-bit models on fold 0 and on RSNA.

Done when (M2, Fri 6 Nov) `reports/xray_fusion.json` holds both comparisons.

S3 Test-time adaptation (Mon 2 to Sat 7 Nov)

Aayush (proposal 4.3)

☐ Switch to Mihir's model. `lung/eval/run_tta.py`:

- N = 8, 16, 32, 64, 128 images, with 5 random draws each;
- shifts: RSNA, and each corruption at severity 3;
- the N images used to adapt are never scored;
- recovery = (adapted − plain) / (clean − plain), on accuracy and AUROC;
- one-sided batches: draws with 10%, 50% and 90% pneumonia;
- time to adapt on a laptop CPU.

Output: `reports/xray_tta.json`.

☐ Optional, only if the above is done by Thu 5 Nov: entropy minimisation as a second method.

S4 8-bit model, robustness and edge budget (Sat 7 to Tue 10 Nov) → M3

☐ **Mihir:** re-quantise the adapted model, using the N clinic images as the representative set; they need no labels. Record agreement with the adapted float model and the conversion time.

☐ **Aayush:** edge budget on one ordinary laptop, for the original and the adapted 8-bit models. Output: `reports/xray_edge_budget.json` with:

- file size;
- agreement with the float model on fold 0;
- time per X-ray and peak memory;
- time for one whole case: CNN, characteristics, occlusion map, votes and fusion.

☐ **Vedansh:** plain against adapted accuracy for all four corruptions at all five severities. Output: `reports/xray_robustness.json`.

☐ **Harsh:** add the adapted model to the generalisation comparison, which is the full CardioNary branch.

Done when (M3, Tue 10 Nov) all three files exist and are checked against Section 5.

S5 Code complete and results frozen (Wed 11 to Mon 16 Nov) → M4

All code ends on Mon 16 Nov, this team's part of the dashboard included. The write-ups in S6 start on Fri 13 Nov and run alongside.

☐ **Mihir, by Fri 13 Nov:** commit one matching set of model files from a single `core.quantize` run of the final model: the float model, the 8-bit model and `reports/quantization_chiron_pulmo_v1.json`. On `main`, the 8-bit file came from a different run than the float model and the report.

☐ **Harsh, by Fri 13 Nov:** the fitted numbers the dashboard needs.

- Save them in `reports/xray_fusion_params.json` as plain numbers: T, the weights, and each vote's column order, filling values, scaler and coefficients.
- Write `xray_fusion(p_cnn, features, metadata)`, which uses only that file and returns a `FusedOutput("xray", ...)`; p_{cnn} is the calibrated pneumonia probability.

☐ **Vedansh, by Sat 14 Nov:** `lung/predict.py`, the X-ray result for the dashboard (Section 3 of the frontend guide):

- `analyse_xray(image, metadata)` takes a PNG, JPEG or DICOM image, as a path or from memory (`io.BytesIO`), and a `PatientMetadata`;
- DICOM through `pydicom`: keep only the pixels, so no name or ID reaches the dashboard;
- the shipped 8-bit model, then temperature T, then a `ModelOutput` whose secondary values are the characteristics;
- the CLAHE view and the 3 × 3 occlusion map, returned as pictures for the dashboard to show;
- votes and fusion through `xray_fusion`;
- tests in `tests/test_lung_predict.py` on made-up images, so they run without data.

☐ **Vedansh:** change `streamlit==1.66.1` to `1.65.0` in `lung/requirements-lung.txt`. Streamlit 1.66 needs Python 3.11, and the team and the CI use 3.10.

☐ **Vedansh, by Mon 16 Nov:** once the dashboard calls `lung/predict.py`, check its X-ray tab with public images: the same p_x as `analyse_xray`, the CLAHE view and the occlusion map shown, and no DICOM tag on the page. Send any problem to the dashboard's owner.

☐ **Aayush:** run the edge budget again through `lung/predict.py`, so "time for one whole case" is the dashboard's own.

☐ **Harsh:** `lung/run_all.py` regenerates every `reports/xray_*.json` from the caches, in order. Run it once more on Mon 16 Nov, after the last code change, so every committed number comes from the final code.

☐ **Everyone:** open the `lung` → `develop` pull request as soon as `lung/predict.py` is in, so the dashboard can call it, and merge `lung` into `develop` again after the last change on Mon 16 Nov.

Done when (M4, Mon 16 Nov) all X-ray code is merged into `develop`; `analyse_xray` gives an answer for one RSNA image with internet switched off and its tests pass; and `run_all.py` has regenerated every X-ray number from the final code.

S6 Report, slides and deliverables (Fri 13 to Tue 17 Nov) → M5

This stage runs alongside S5 from Fri 13 Nov. Write from the committed result files, and take the final numbers from them after the code freeze on Mon 16 Nov.

☐ Each person writes the report section and makes the slides for their part: the method in a few lines, the results table with intervals, and what did not work.

- Mihir: 4.1 and 8-bit.
- Harsh: 4.2 and calibration.
- Aayush: 4.3 and timing.
- Vedansh: unseen data, robustness and the X-ray result for the dashboard.

☐ **Mihir and Aayush:** check every number in the report and the slides against those files, and that each has its bootstrap interval. Check again on Tue 17 Nov, after the code freeze.

☐ **Vedansh:** a 2 to 3 minute demo video of the dashboard's X-ray tab, recorded on Tue 17 Nov once the code is frozen: offline, with Kermany images (CC BY 4.0). Leave RSNA images out of it: their Kaggle rules limit sharing.

☐ **Everyone:** the AI-usage note (tools used, for what, how the output was checked), and experiment notes in `lung/experiments/`, including failed ones.

Done when (M5, Tue 17 Nov) every deliverable is ready. Nothing new starts after this date.

5. Targets and what we compare

The targets are the X-ray rows of the proposal's success criteria (4.7), fixed in advance.

Goal	Measure	Target	Stage
Base model	Kermany 5-fold per-patient accuracy	≥ 90%	S1
Generalisation	AUROC gain of the full branch over the plain CNN on RSNA	≥ +0.03	S2, S4
Metadata	Referral AUROC gain from the metadata vote on RSNA	≥ +0.02, with $w_3 \leq 0.3$	S2
Adaptation	Share of the accuracy lost on RSNA that is recovered, N ≤ 64	≥ 50%, in under 5 s on a CPU	S3
Calibration	Expected calibration error after temperature scaling	≤ 0.05	S2
Edge budget	8-bit size; agreement with float; time per case	< 1 MB; ≥ 98%; < 1 s on a laptop CPU	S4, S5

What we compare (proposal 4.7).

- Three versions: plain CNN; CNN plus feature vote; CNN plus feature vote plus metadata vote.
- Each version with and without adaptation.
- Datasets: Kermany (5-fold), RSNA, and corrupted Kermany.
- Metrics: AUROC, sensitivity at 90% specificity, per-patient accuracy, expected calibration error (15 bins), model size, memory use and time per case.
- Every result carries a 95% bootstrap interval from 1,000 resamples.

6. Each person's weeks

Member	26 Oct to 1 Nov	2 to 8 Nov	9 to 16 Nov: code	13 to 17 Nov: write-ups
Mihir	Kermany data; base CNN; out-of-fold file; final and 8-bit models	Occlusion maps; 8-bit agreement	Re-quantisation after adaptation; matching model files	Report and slides: data, CNN, 8-bit; checking numbers
Harsh	CLAHE; characteristics; feature vote; calibration code	Calibration; metadata vote; late fusion	Adaptation in the comparison; fitted numbers for the dashboard; `run_all.py`	Report and slides: characteristics, votes, fusion, calibration
Aayush	Own model copy; adaptation code; timing script	Adaptation experiments	Edge budget, again through `lung/predict.py`	Report and slides: adaptation, timing; checking numbers
Vedansh	RSNA data; metrics; corruptions	RSNA runs; the two comparisons	Robustness; `lung/predict.py`; checking the dashboard's X-ray tab	Report and slides: unseen data, robustness, X-ray result for the dashboard; demo video

7. How we work, rules and risks

Routine.

- A 10-minute check-in every evening.
- Make your branch from `lung` as `lung-<name>-<topic>`, open a pull request back into `lung`, and have one teammate approve it. Merge with "Create a merge commit".
- If someone is unavailable, Mihir and Aayush cover each other (model and edge), and so do Harsh and Vedansh (fusion and evaluation).

Rules that keep the numbers honest.

- Folds are patient-disjoint.
- Normalisation, standardisation and temperature are fitted on training folds only.
- Nothing is fitted on the fold being scored.
- The images used for adaptation are never scored.
- Every reported number comes from a script and a committed JSON file.
- Seeds are fixed at 0.
- The dashboard fits nothing. T, the weights and the vote coefficients come from committed files.

Risk	What we do (easiest first)
Base CNN below 90% per patient	Check the cache and normalisation; make sure `--augment` was passed; then unfreeze more layers (lower `trainable_from` in `build_mobilenet`)
RSNA download slow or blocked	Start it on day 1 on the fastest connection; if needed, use a stratified sample of 3,000 Normal and 3,000 Lung Opacity images and say so
Metadata vote learns a shortcut	Only age and sex go in, its weight is capped at 0.3, and view position is excluded; report its weight and its AUROC on its own

Risk	What we do (easiest first)
Feature vote adds little on RSNA	Accept a small weight and report it; the characteristics still appear in the dashboard and report
Adaptation hurts on small or one-sided batches	Use N = 32 by default; the collapse guard keeps the original statistics; report the failing settings
8-bit agreement below 98%	Raise the representative set from 60 to 200 images per class in your copy of the call
A laptop is too slow	Run the same commands in Google Colab; RSNA files go only to accounts that accepted the Kaggle rules
The dashboard is late	`analyse_xray` and its tests stand on their own, so the X-ray branch is complete without it, and no X-ray result in the report depends on the dashboard
Running out of time	Drop in this order: entropy minimisation, two of the four corruption types, the occlusion map in the dashboard. Never drop freezing the results or `lung/predict.py`

Shared with the heart team. The heart team builds the dashboard, joins the two branches and makes the visit decision (proposal 4.4). This team's part is the hand-off: `lung/predict.py`, the fitted numbers and matching model files, through `develop` by Sat 14 Nov, then checking the dashboard's X-ray tab by Mon 16 Nov, when all code ends.